

Low-cost real-time electronic line caller for volleyball

Bradley Hong
10/3/26

Abstract

Calling balls in or out for volleyball can be difficult, especially since most line callers are untrained students, so a low-cost real-time electronic line caller was designed, constructed, and tested to try to address this need. The prototype used less than \$500 of components and consisted of a Raspberry Pi, high-speed USB camera, and computer vision algorithms in OpenCV python. The algorithms detect which pixels change in each frame to detect balls. During testing, the prototype demonstrated 47 out of 48 correct line calls with volleyballs hit in a gym.

Introduction

The current system for calling balls in or out in volleyball has many problems. Oftentimes, volleyballs travel very fast, or they can go towards the line referees' feet, making correct calls difficult. In high school volleyball, the line callers are students and players, so they are often untrained. They might be not paying attention or, worse, be biased toward the home team. Bad line calls can shift the momentum or outcome of a game, so inaccurate line reffing is not something to be taken lightly.

Electronic systems solve this problem, but those can cost close to \$100,000 [1] per court, unaffordable for high schools and community centers. Some examples of them are Hawk-Eye [2] and Bolt6 [3]. The Bolt6 requires 20 cameras and an operator, technician, and a specialized ref to run, use, and analyze these systems [4].

There have been attempts to make low-cost systems. For example, White [5] made a low-cost prototype of a line caller as a capstone project at Union College. It used sensors on the ground that detected balls and the length of the signal to differentiate a ball and a person. In addition, some commercial line-calling products exist that use video replay when calls are challenged, but this interrupts the flow of the game and requires the ref to have a screen.

This project aims to demonstrate the feasibility of a low-cost real-time electronic volleyball line-calling system that could be practical for schools and community centers.

Goals

First, the prototype should be affordable for communities and high schools. This will make the system more accessible, as systems like hawkeye are too expensive.

Hardware

A Raspberry Pi 5 with 8GB of RAM was used as the main computer. A camera mounted on a low tripod recorded footage. A rechargeable portable battery powered it all. The current battery has 10,000 mAh and the pi consumed at max 12 W, so the battery could power it for 4 hours at a typical power consumption at 8 W, or 1 hour at the maximum power consumption of 25 W.

Figure 1: Photo of prototype line-calling system.

Component	Model	Cost
Raspberry Pi	CanaKit Raspberry Pi 5 Starter Kit PRO (128GB Edition) (8GB RAM)	\$259.95
Camera	ELP High Speed USB Zoom Camera, 1080P/230fps, 4K/60fps, 12MP/30fps, USB3.0/2.0 Manual Focus Lightburn Cameras for Windows, Linux, Mac, Pi, Fast Frame 5-50mm 10X Zoom IMX577 Webcam	\$148.99
Tripod	Zeadio Heavy Duty Metal Tripod Kit, Universal 1/4" Screw Adapter Tripod	\$38.98
Battery	Anker Portable Charger, USB-C PortableCharger 10000mAh with 20W Power Delivery, 523 Power Bank	\$27.99
LED	7 Bits RGB LED Rings 7 X WS2812B 5050 Lamp Light	\$6.69

Table 1: List of components used for prototype line-calling system.

Computer Vision

The algorithm for calling the ball uses the absolute difference function of OpenCV. First, it takes a small clip around half of a second. Then, it converts the pixels to absolute difference. This means that for each pixel, if the same pixel between the previous frame and this frame changed, the pixel will be brighter. Otherwise, it will be dark. If there's a certain number of pixels changed, the video will detect motion. A red circle of the ball is placed by the mean of the x and

y positions of all the changed pixels and a hard-coded diameter value. Every frame, the red circle is drawn if enough pixels are changed. The program tracks the x and y values of each frame with movement. Since the ball is bouncing, the point of impact is shown by the lowest y position in the list. Then, it compares the x position corresponding to that y position, calling in or out by how far right or left (depending on the ball direction) of a hard-coded line by another hard-coded value representing the ball squish on the floor.

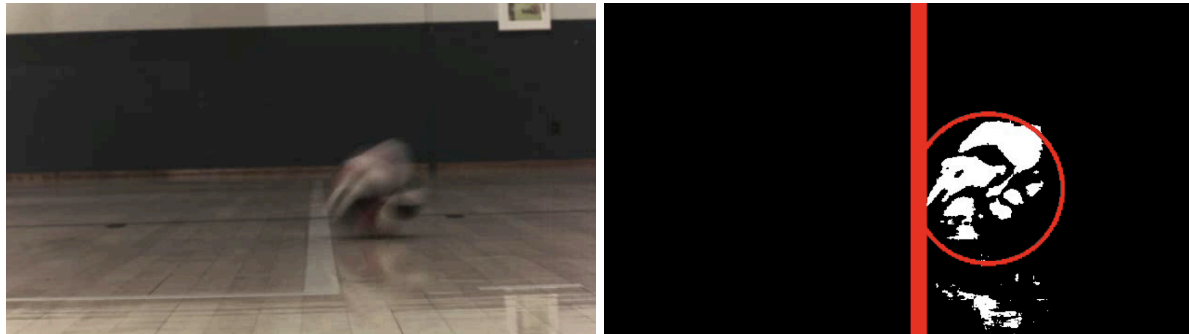


Figure 2: Frame of a ball in motion. Left: Raw video. Right: Processed frame with the red line indicating the end line of the court and the circle indicating the position of the ball.

Results

The prototype system was tested by going to a gym with a wooden floor at night with only the gym's regular lighting. The camera was aimed at one base line about 10ft away and recorded a 5-minute clip while a player spiked a Molten V5M5000-3USA ball on the base line. The frame rate was about 103fps. After the recording, the video was broken into clips and processed with a computer vision algorithm. Then, a person would go through a slow-mo version of the clips and call them manually.

The performance was determined by whether the human agrees with the prototype's call. This [table](#) shows the results for each clip. As shown, the call was correct 47/48 times, or 97% correct. It was almost always accurate, but one clip where it didn't agree was clip 84. Not only was it close to the line, but the ball was further away. Since the ball diameter was larger than it should be due to the ball being further away from the camera, it thought that the ball that was out was touching the line, so it called it in.

	Computer In	Computer Out
Human In	26	0
Human Out	1	21

Figure 2: The amount of calls for each individual case.

Limitations and future improvements

One glaring issue with using a single camera system is that a player could cover the camera, not only registering motion, but also blocking the machine from processing the ball. A future improvement would be using the frames it does see to predict the ball movement. Such a system could be unpredictable, but could withstand lots of distractions. Another solution is to use two or more cameras. However, not only both cameras could still be blocked, but it would be twice as expensive. In addition, there are many hard coded values that only work for the specific camera settings and ball placement. A great improvement will be making a system that automatically makes those values according to the camera feed. Another limitation is all the motion in the audience, as well as the shadows from the ball. A solution would be to make a new algorithm for a version of it pointing down, but that would mean that detecting the vertical position wouldn't work anymore. A more minor issue is that the call is delayed about 3 seconds after the ball lands. This can potentially be fixed by changing the clip time. If the hardware were to take a hit from a ball, it could break the program. A future improvement could be to design a shock-resistant case for the hardware.

There are multiple cases where the cost could be reduced. First, a Raspberry Pi with 8GB of RAM isn't necessary for this task. A less expensive computer with less computing capacity would do the job. The camera used was a 12 megapixel, and that image quality isn't necessary. Using a cheaper camera would be fine. Another potential solution would be to design a camera to look both ways. This would half the cost, but would require much more time to make an algorithm.

Conclusions

This project showed that it's feasible to have a low cost system for assisting with volleyball line calls. It could either assist the human line callers or replace them all together and assist the head referee instead. While improvements would need to be made before use in live matches, the testing showed that the prototype system is already fairly accurate for calling lines. The sport of volleyball would greatly benefit from devices like these, and hopefully they can become affordable outside the professional leagues.

References

1. <https://huddleup.substack.com/p/the-100000-camera-system-thats-quietly>
2. <https://www.hawkeyeinnovations.com>
3. <https://www.bolt6.ai/>
4. <https://www.youtube.com/watch?v=h3BZ4do66vM&feature=youtu.be>
5. <https://bpb-us-w2.wpmucdn.com/muse.union.edu/dist/6/356/files/2016/03/FINAL-PAPER-499.pdf>